

RAIMES Testing Documentation

4.1 Introduction

4.1.1 Scope

4.1.1.1 In Scope

- Authentication API testing (login, registration)
- File upload testing (images, documents, evidence)
- Assessment API testing (create, submit, results)
- Admin functionality testing
- Image upload endpoints testing

4.1.1.2 Out of Scope

- Frontend UI testing
- Database performance testing
- Third-party integrations

4.1.2 Quality Objectives

- API response time < 2 seconds
- File upload success rate > 95%
- Zero security vulnerabilities
- All endpoints return proper HTTP status codes

4.1.3 Roles and Responsibilities

- **QA Tester:** Execute test cases using Postman
- **Developer:** Fix bugs and provide API documentation
- **Admin:** Validate admin-specific functionalities

4.2 Test Methodology

4.2.1 Overview

Testing will be conducted using Postman for API endpoint validation and file upload testing.

4.2.2 Test Levels

4.2.2.1 Unit Testing

- Individual API endpoint testing
- Input validation testing
- Error handling verification

4.2.3 Bug Triage

- **Critical:** Authentication failures, data loss
- **High:** File upload failures, incorrect responses
- **Medium:** Performance issues, UI inconsistencies
- **Low:** Minor text errors, formatting issues

4.2.4 Suspension Criteria and Resumption Requirements

- **Suspend:** If authentication system fails completely
- **Resume:** After critical bugs are fixed and verified

4.2.5 Test Completeness

- All endpoints tested with valid/invalid inputs
- File upload tested with various formats and sizes
- Authentication flow completely verified

4.3 Resource & Environment Needs

4.3.1 Testing Tools

4.3.1.1 Postman

Primary tool for API testing and file upload validation.

Base URL: http://localhost:5000

4.4 Terms/Acronyms

- **API:** Application Programming Interface
- **JWT:** JSON Web Token
- **CRUD:** Create, Read, Update, Delete

API Endpoints for Testing

Authentication Endpoints

Login

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "admin@raimes.com",
  "password": "password123"
}
```

Register Request

```
POST /api/auth/register-request
Content-Type: application/json

{
  "companyName": "Test Company",
  "email": "test@company.com",
  "password": "password123"
}
```

File Upload Endpoints

Upload Website Screenshot

```
POST /api/company/upload-website-screenshot
Authorization: Bearer {token}
Content-Type: multipart/form-data

Fields:
- screenshot: File
- description: "Homepage screenshot"
- pageType: "homepage"
```

Upload Company Logo

```
POST /api/company/upload-logo
Authorization: Bearer {token}
Content-Type: multipart/form-data

Fields:
- logo: File
- description: "Company logo"
```

Upload Evidence Photo

```
POST /api/evidence/upload-photo
Authorization: Bearer {token}
Content-Type: multipart/form-data

Fields:
- photo: File
- assessmentId: "123"
- questionId: "456"
- description: "Certificate evidence"
```

Assessment Endpoints

Create Assessment

```
POST /api/assessment/create
Authorization: Bearer {token}
Content-Type: application/json

{
  "questionnaireId": 1,
  "companyId": 123
}
```

Get Assessment Results

```
GET /api/assessment/{id}/results
Authorization: Bearer {token}
```

Admin Endpoints

Get Registration Requests

```
GET /api/admin/registration-requests
Authorization: Bearer {admin_token}
```

Approve Registration

```
PUT /api/admin/registration-requests/{id}/approve
Authorization: Bearer {admin_token}
Content-Type: application/json

{
  "role": "user"
}
```

Questionnaire & AI Engine Endpoints

Get Questionnaire by ID

```
GET /api/questionnaire/{id}
Authorization: Bearer {token}
```

Submit Questionnaire Answer

```
POST /api/questionnaire/{id}/submit-answer
Authorization: Bearer {token}
Content-Type: application/json

{
  "questionId": 123,
  "answer": "Yes",
  "evidenceFiles": ["cert.pdf", "photo.jpg"],
  "explanation": "We have implemented ISO 27001"
}
```

AI Evidence Processing

```
POST /api/ai/process-evidence
Authorization: Bearer {token}
Content-Type: multipart/form-data

Fields:
- file: File (PDF, Image)
- questionId: "456"
- assessmentId: "789"
```

AI Validation Service

```
POST /api/ai/validate-answer
Authorization: Bearer {token}
Content-Type: application/json

{
  "questionId": 123,
  "answer": "Yes",
  "evidence": ["file1.pdf", "file2.jpg"],
  "explanation": "Detailed explanation"
}
```

AI Scoring Service

```
POST /api/ai/calculate-score
Authorization: Bearer {token}
Content-Type: application/json

{
  "assessmentId": 456,
  "answers": [
    {
      "questionId": 1,
      "answer": "Yes",
      "weight": 5,
      "evidenceScore": 0.8
    }
  ]
}
```

Test Cases Summary

Endpoint	Method	Expected Status	Test Type
/api/auth/login	POST	200	Authentication
/api/questionnaire/{id}	GET	200	Questionnaire
/api/questionnaire/{id}/submit-answer	POST	201	Answer Submission
/api/ai/process-evidence	POST	200	AI Processing
/api/ai/validate-answer	POST	200	AI Validation
/api/ai/calculate-score	POST	200	AI Scoring
/api/company/upload-logo	POST	200	File Upload
/api/evidence/upload-photo	POST	200	Evidence Upload
/api/assessment/create	POST	201	Assessment
/api/admin/registration-requests	GET	200	Admin Panel

File Upload Testing

Valid File Formats

- Images: JPG, PNG, GIF
- Documents: PDF, DOC, DOCX
- Max Size: 10MB

Test Scenarios

- Valid Upload:** Upload JPG image < 5MB
- Size Limit:** Upload file > 10MB (should fail)
- Invalid Format:** Upload .txt file (should fail)
- Missing Auth:** Upload without token (should fail)

Security Analysis & Endpoint Safety

🔒 Security Status by Endpoint

Endpoint	Auth Required	Role Check	File Validation	Safety Level
/api/auth/login	✗	✗	✗	✅ SAFE
/api/auth/register-request	✗	✗	✅	⚠️ MEDIUM
/api/questionnaire/{id}	✅	✗	✗	✅ SAFE
/api/questionnaire/{id}/submit-answer	✅	✗	✅	✅ SAFE
/api/ai/process-evidence	✅	✗	✅	⚠️ MEDIUM
/api/ai/validate-answer	✅	✗	✗	✅ SAFE
/api/ai/calculate-score	✅	✗	✗	✅ SAFE
/api/company/upload-logo	✅	✗	✅	✅ SAFE
/api/evidence/upload-photo	✅	✗	✅	✅ SAFE
/api/assessment/create	✅	✗	✗	✅ SAFE
/api/admin/registration-requests	✅	✅	✗	✅ SAFE

🚨 Security Concerns

HIGH RISK

- File Upload Endpoints:** Potential malicious file uploads
- AI Processing:** May process untrusted content
- Public Registration:** No rate limiting visible

MEDIUM RISK

- Missing Role Authorization:** Some endpoints lack role-based access
- File Size Limits:** 10MB limit may allow DoS attacks
- Input Validation:** Need SQL injection protection

LOW RISK

- **JWT Implementation:** Appears secure with proper verification
- **Authentication Flow:** Standard implementation

 Security Testing Checklist

Authentication & Authorization

- ☐ Test login with valid credentials
- ☐ Test login with invalid credentials
- ☐ Test expired JWT tokens
- ☐ Test malformed JWT tokens
- ☐ Test missing Authorization header
- ☐ Test role-based access (admin vs user)
- ☐ Test privilege escalation attempts

File Upload Security

- ☐ Test valid file formats (JPG, PNG, PDF)
- ☐ Test invalid file formats (.exe, .bat, .php)
- ☐ Test file size limits (>10MB)
- ☐ Test malicious file uploads (virus, malware)
- ☐ Test path traversal attacks (.././etc/passwd)
- ☐ Test filename injection attacks
- ☐ Test MIME type spoofing

Input Validation

- ☐ Test SQL injection in all input fields
- ☐ Test XSS attacks in text inputs
- ☐ Test NoSQL injection (if using NoSQL)
- ☐ Test command injection in file processing
- ☐ Test buffer overflow with large inputs
- ☐ Test special characters handling

AI Engine Security

- ☐ Test AI processing with malicious PDFs
- ☐ Test AI with oversized images
- ☐ Test prompt injection attacks
- ☐ Test evidence tampering detection
- ☐ Test AI response manipulation

Rate Limiting & DoS

- ☐ Test multiple rapid login attempts
- ☐ Test bulk file uploads
- ☐ Test concurrent AI processing requests
- ☐ Test large payload attacks
- ☐ Test connection exhaustion

Data Security

- ☐ Test sensitive data exposure in responses
- ☐ Test data encryption at rest
- ☐ Test secure data transmission (HTTPS)
- ☐ Test user data isolation
- ☐ Test audit trail completeness

 Critical Security Recommendations

1. **Add Rate Limiting:** Implement on login and file upload endpoints
2. **Enhanced File Validation:** Add virus scanning and content analysis
3. **Role-Based Access:** Add proper role checks to sensitive endpoints
4. **Input Sanitization:** Implement comprehensive input validation
5. **Audit Logging:** Add security event logging
6. **HTTPS Enforcement:** Ensure all traffic is encrypted
7. **CORS Configuration:** Properly configure cross-origin requests
8. **SQL Injection Prevention:** Use parameterized queries
9. **File Storage Security:** Isolate uploaded files from execution paths
10. **AI Input Validation:** Sanitize all AI processing inputs